Hyperledger Fabric Node.js Chaincode Learning & Testing Roadmap

---

Section 1: Node.js Fundamentals (Days 1–5)

1. Variables & Constants

```
let count = 0;
const name = 'Hyperledger';
```

2. Data Types - String, Number, Boolean, Object, Array

3. Functions

```
function greet(name) { return `Hello, ${name}`; }
const add = (a,b) => a+b;
```

4. Async/Await

```
async function fetchData() { let data = await someAsyncCall(); return data; }
```

5. Classes

```
class Project {
  constructor(id, title){ this.id = id; this.title = title; }
  getTitle(){ return this.title; }
}
```

Exercises: Classes, async functions, objects, arrays.

Section 2: Fabric Chaincode Basics (Days 6–10)

1. Contract Class

```
const { Contract } = require('fabric-contract-api');
class MyChaincode extends Contract { constructor(){ super('MyChaincode'); } }
module.exports = MyChaincode;
```

2. Context (`ctx`) - `ctx.stub` → ledger operations (`putState`, `getState`) - `ctx.clientIdentity` → access user info

3. Ledger Operations

```
await ctx.stub.putState(key, Buffer.from(JSON.stringify(object))); // store
const data = await ctx.stub.getState(key); // read
await ctx.stub.deleteState(key); // delete
```

4. Chaincode Transaction Functions - Async methods inside chaincode class - Invokable by client SDK

Exercises: Create, read, delete asset methods.

Section 3: Advanced Ledger Operations (Days 11–15)

1. Query All Assets

```
async queryAllAssets(ctx){
  const iterator = await ctx.stub.getStateByRange('', '');
  const results = [];
  for await (const res of iterator) {
    results.push(JSON.parse(res.value.toString('utf8')));
  }
  return results;
}
```

2. Error Handling

```
if(!assetBytes || assetBytes.length===0) throw new Error('Asset not found');
```

3. Events

```
ctx.stub.setEvent('EventName', Buffer.from(JSON.stringify(payload)));
```

4. Composite Keys

```
const key = ctx.stub.createCompositeKey('Project', [studentName, projectTitle]);
await ctx.stub.putState(key, Buffer.from(JSON.stringify(project)));
```

Exercises: Implement `queryAllAssets`, composite keys, events.

Section 4: Node.js SDK for Testing Chaincode (Days 16–20)

1. Gateway - Connects app to Fabric network

```
const gateway = new Gateway();
await gateway.connect(ccp, { wallet, identity: 'appUser', discovery:
{enabled:true, asLocalhost:true} });
```

2. Wallet - Stores identities

```
const wallet = await Wallets.newFileSystemWallet('./wallet');
```

3. Network & Contract

```
const network = await gateway.getNetwork('mychannel');
const contract = network.getContract('ProjectChaincode');
```

4. Transactions

```
await
contract.submitTransaction('uploadProject','P1','Alice','SmartCar','Autonomous
project');
const result = await contract.evaluateTransaction('queryProject','P1');
console.log(result.toString());
```

- `submitTransaction` → modifies ledger - `evaluateTransaction` → read-only

Exercises: Test CRUD methods via SDK, handle errors.

Section 5: Expert Chaincode Concepts (Days 21+)

1. Private Data Collections
2. Access Control via `ctx.clientIdentity`
3. Rich Queries with CouchDB
4. Events for testing and triggers
5. Code structuring with helper classes/functions

Exercises: Restrict update methods, trigger events, implement private data fields.

Section 6: Testing Chaincode

1. Fabric CLI

```
cd fabric-samples/test-network
./network.sh up
./network.sh createChannel -c mychannel
peer lifecycle chaincode install mycc.tar.gz
peer lifecycle chaincode approveformyorg ...
peer lifecycle chaincode commit ...
peer chaincode invoke -C mychannel -n mycc -c '{"function":"createAsset","Args":["A1","100"]}'
peer chaincode query -C mychannel -n mycc -c '{"Args":["readAsset","A1"]}'
```

- Check ledger files for updates - Validate JSON outputs and error messages

2. Node.js SDK Testing

1. Setup Gateway & Wallet
2. Connect Network & Contract
3. Submit transactions (`submitTransaction`) for ledger write
4. Evaluate transactions (`evaluateTransaction`) for queries
5. Test errors: duplicate entries, missing assets
6. Iterate with multiple projects

3. Best Practices

- Start with empty ledger for testing

- Console logs inside chaincode for debug
- Test CRUD individually
- Test error handling and invalid inputs
- Use SDK scripts to simulate multiple users

Daily Workflow for Coding & Testing

1. Read concept (variable, function, class, ctx, putState etc.)
2. Write chaincode snippet
3. Test using CLI and SDK
4. Document results for FYP notes